

# PROJECT HADES

Engineering Bible v1.0

## Chapter 4 — System Architecture

Define the complete internal architecture of Hades.

### Goal

This chapter specifies the components, responsibilities, data flow, execution pipeline, and interaction between every major subsystem. It is the blueprint from which the engineering team can implement Hades.

### Proposed Structure (≈5 Pages)

| Page | Focus                  | What It Must Cover                                                     |
|------|------------------------|------------------------------------------------------------------------|
| 1    | Architecture Overview  | Layering, philosophy, unified experience, high-level architecture      |
| 2    | Core Components        | Partner Brain, Executive Brain, Mission Engine, their responsibilities |
| 3    | Workers & Tools        | Worker lifecycle, tools, execution boundaries                          |
| 4    | Memory & Review Engine | Memory layers, validation, retry logic                                 |
| 5    | Data Flow              | End-to-end flow and engineering principles                             |

# Page 1 — Architecture Overview

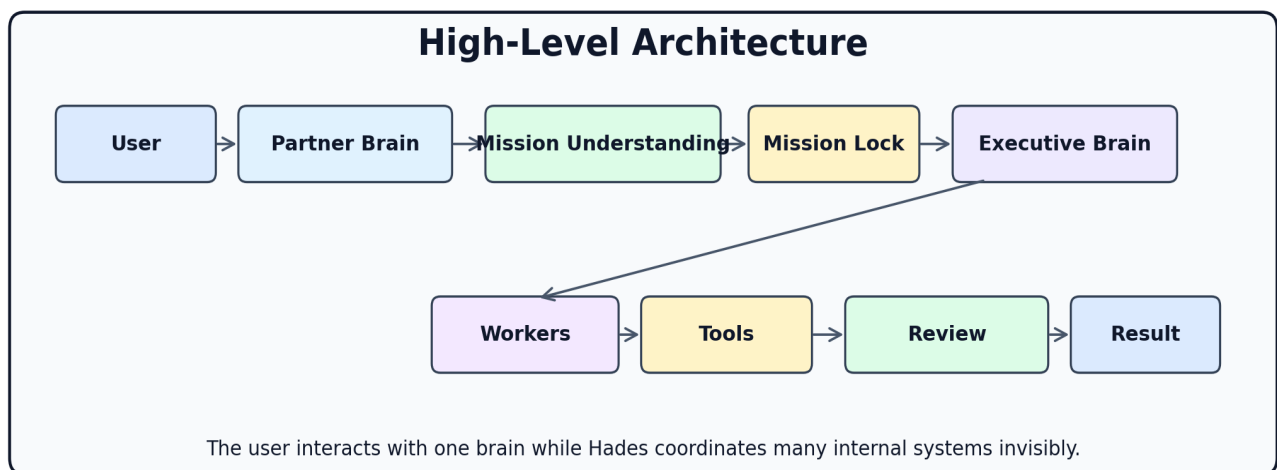
## Chapter Overview

Hades is not a single AI model. It is a coordinated intelligence composed of multiple specialized layers that work together while presenting a single, unified experience to the user.

## Architecture Philosophy

- Hide internal complexity.
- Expose only one persistent intelligence.
- Separate conversation from execution.
- Be model-agnostic.
- Support future expansion.

## High-Level Architecture



The user interacts with **one brain**, while Hades coordinates many internal systems invisibly.

## Page 2 — Core Components

### Partner Brain

Responsibilities: conversation, reasoning, clarification, memory retrieval, alignment, and mission creation.

Inputs: user conversation, memory, active missions.

Outputs: clarified mission, questions, mission lock decision.

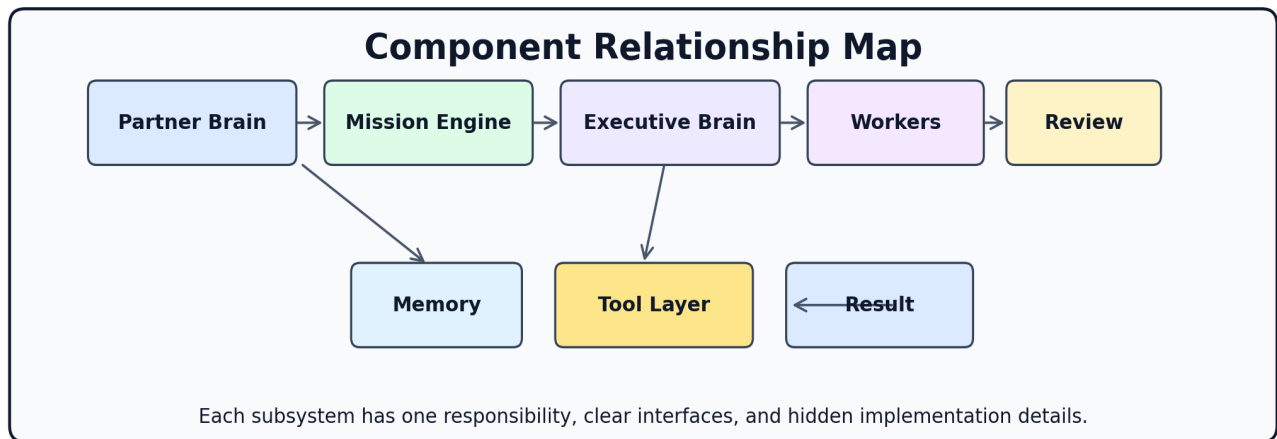
### Executive Brain

Responsibilities: planning, delegation, worker selection, tool orchestration, progress monitoring, and recovery.

### Mission Engine

Responsible for mission lifecycle, priorities, dependencies, constraints, success criteria, and mission state transitions.

### Component Relationship Diagram



## Page 3 — Workers & Tools

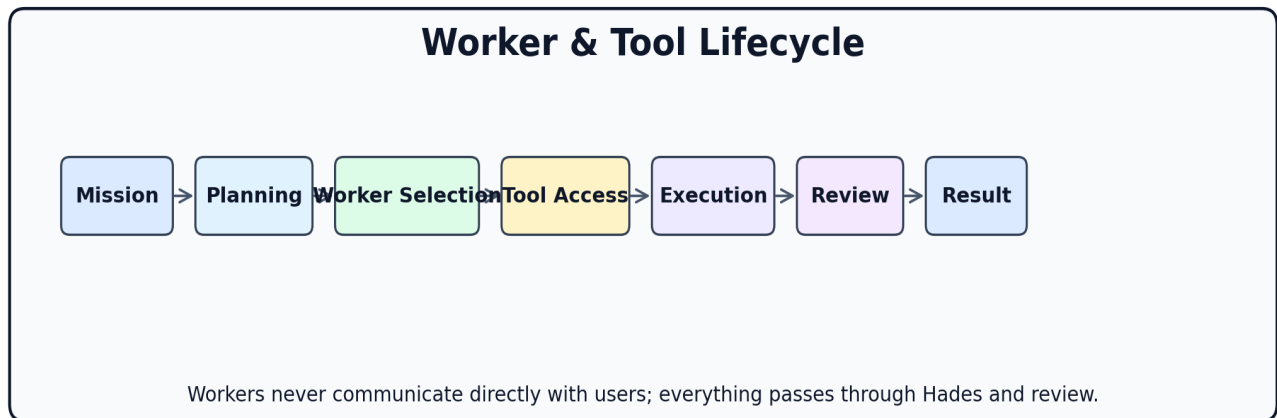
### Workers

Workers are specialized execution capabilities such as coding, research, writing, design, planning, and analysis. Workers are implementation details. Users never choose workers.

### Tool Layer

- Browser
- Terminal
- Git
- File System
- VS Code
- APIs
- Email
- Calendar

### Worker Lifecycle



Workers never communicate directly with users. All outputs pass through Hades.

## Page 4 — Memory & Review Engine

### Memory Layers

**Session Memory** - current conversation.

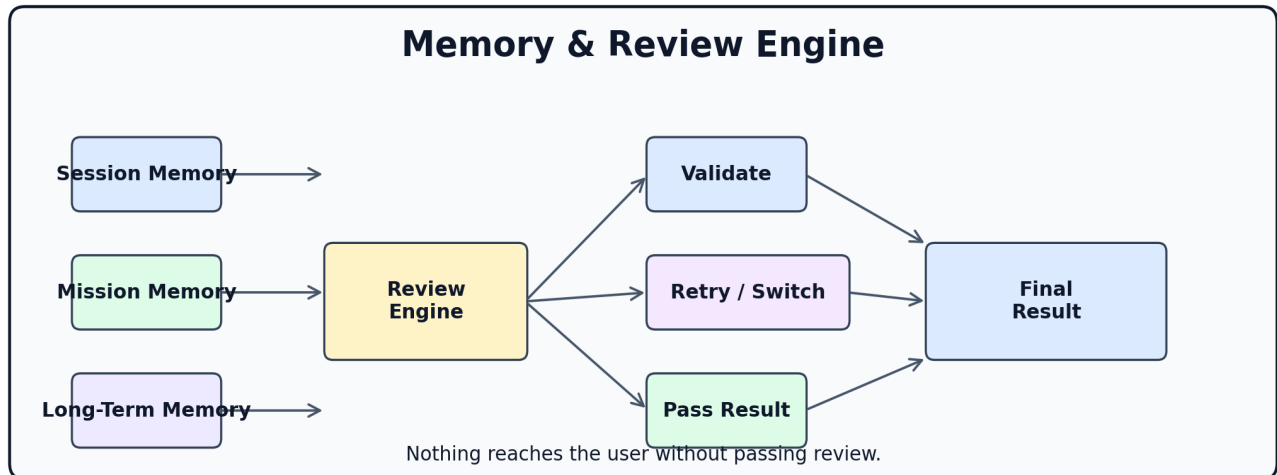
**Mission Memory** - mission state.

**Long-Term Memory** - projects, preferences, goals, history, lessons learned.

### Review Engine

Every worker output is validated for correctness, completeness, safety, alignment, and mission requirements.

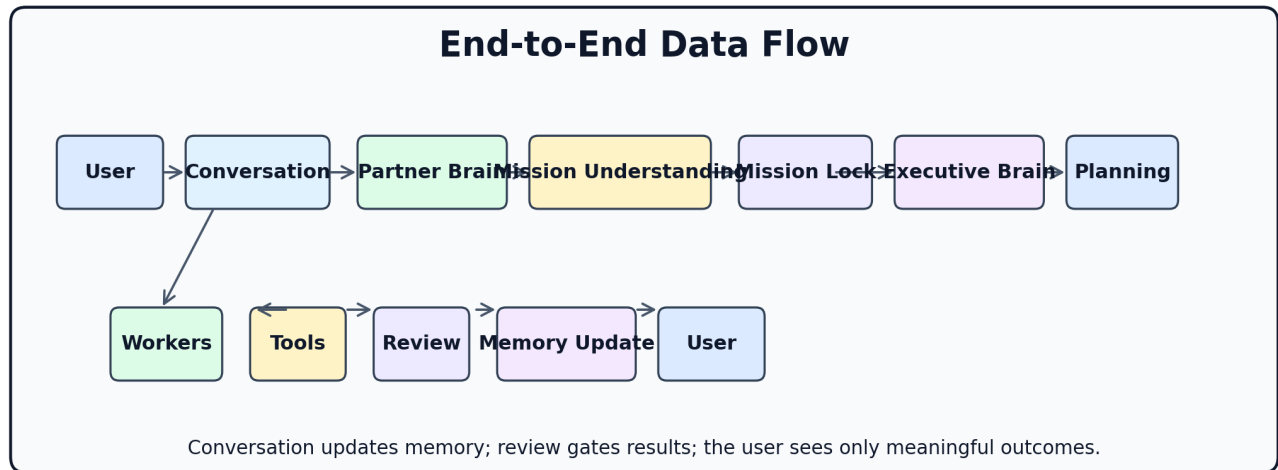
### Retry Logic



Nothing reaches the user without passing review.

## Page 5 — Data Flow

### Complete Flow



### Engineering Principles

- Every module has one responsibility.
- Components communicate through well-defined interfaces.
- Internal complexity is hidden.
- Models are replaceable.
- Memory is centralized.
- Review is mandatory.
- Conversation precedes execution.

### Future Architecture

The architecture should support local models, cloud models, multi-device synchronization, autonomous scheduling, plugin ecosystem, custom workers, and distributed execution.

### Visuals Included

- High-level architecture diagram
- Component relationship map
- Mission pipeline
- Worker lifecycle flowchart
- Memory architecture diagram
- Review engine pipeline
- End-to-end execution flow
- Layered system architecture
- Responsibility matrix
- Component interaction map